ChatGPT

Executive Summary

This report presents a **comprehensive, multi-session C++ curriculum** designed to take a complete novice through all fundamental and advanced topics in modern C++. Across **20+ progressive sessions**, learners will cover everything from the very basics ("what is a program?", compilation, execution) to advanced concepts (modern language features, templates, metaprogramming, concurrency, and data structures/algorithms). Each session is **self-contained** but builds on prior material. It includes clear learning objectives, concise explanations, annotated code examples, common pitfalls, exercises (5 per session, labeled *Easy/Medium/Hard*), and a mini-project or challenge.

Special emphasis is given to **program flow and runtime details** (program startup/termination, linking, stack vs heap, object lifetimes, undefined behavior, diagnostics). Modern C++ features (C++11/14/17/20/23) are covered with notes on how they map to older idioms and advice for reading legacy code. We also recommend tools (compilers, build systems, debuggers, IDEs, sanitizers) with cross-platform usage and commands. Progress is tracked via periodic checkpoints and quizzes. Authoritative sources (ISO C++ standards, *cppreference*, and key references) are cited throughout. Visual aids include mermaid diagrams of memory layout, compilation pipeline, and a C++ standards timeline.

Below is a **session-by-session table** summarizing all 20+ planned sessions.

Session	Title	Length	Prerequisites
1	Introduction & Setup: "Hello, World!"	60 min	None
2	Variables, Types & I/O	60 min	Session 1
3	Control Flow (if, loops, switch)	60 min	Session 2
4	Functions & Program Flow (stack, call graph)	60 min	Session 3
5	Pointers & References	60 min	Session 4
6	Dynamic Memory & Object Lifetime (new, delete)	60 min	Session 5
7	Arrays, C-Strings & `std::string`	60 min	Session 5
8	Classes & Objects (constructors, destructors)	60 min	Session 6
9	Inheritance & Polymorphism (virtual, override)	60 min	Session 8
10	Templates & Generic Programming	60 min	Session 8
11	STL Basics: `std::vector`, `std::list`, Iterators	60 min	Session 8
12	STL Algorithms & Containers (`map`, `set`, etc.)	60 min	Session 11
13	Smart Pointers & Move Semantics	60 min	Session 6

Session	Title	Length	Prerequisites
14	Modern C++ Features (lambdas, auto, ranges)	60 min	Sessions 2–6
15	Concurrency Basics (threads, mutex, atomics)	60 min	Sessions 5, 13
16	Concurrency Advanced (futures, async, thread-safety)	60 min	Session 15
17	Metaprogramming (constexpr, SFINAE, Concepts)	60 min	Sessions 10, 14
18	Data Structures I (complexity, containers)	60 min	Sessions 11–12
19	Algorithms I (sorting, searching, BFS/DFS)	60 min	Session 18
20	Algorithms II (graphs, DP, greedy)	60 min	Sessions 18–19
21	Build Systems & Tools (CMake, Debugging, etc.)	60 min	All above
22	Final Project & Review; Assessment	60 min	All above

Each session below details objectives, key concepts, example code (with comments), pitfalls, exercises, and mini-project. **Primary sources** are cited (e.g. C++ standard papers, *cppreference*, and authoritative guides) to ensure accuracy.

Session 1: Introduction & Setup — "Hello, World!"

Objectives: Understand what a program is, learn the basic structure of a C++ program, and run a first "Hello, world!" program. Learn the compilation pipeline and how to use compilers on different platforms.

Key Concepts: Source code ↔ object code ↔ executable; compilation phases (preprocessing, compiling, assembling, linking); basic I/O via `<iostream>`; `main()` as entry point; comment syntax; simple compilation commands.

```cpp
// hello.cpp
#include <iostream>            // include I/O library
int main() {
    std::cout << "Hello, World!" << std::endl;  // print message
    return 0;                // return success code
}
```

- **Annotations:** `#include <iostream>` gives access to `std::cout` for printing. The function `main()` is the program entry point [1]. We output to the console and return 0.

- **Pitfalls:** Missing `;` causes compile errors. Forgetting `return 0;` is allowed (C++ specifies `return 0;` by default at end of `main`). Using `std::cout` requires `#include <iostream>`.

- **Compilation (Linux/Mac):**

```
g++ -std=c++23 hello.cpp -o hello    // compile with C++23 standard
./hello                              // run executable
```

(On Windows with MSVC: `cl /EHsc hello.cpp`; on Linux, Clang: `clang++ -std=c++23 hello.cpp -o hello`.)

- The compiler translates source to object code (often `.o`), then the linker combines objects into an executable [2] [3].

- **Memory/Layout Diagram:**

```
graph LR
  Code["Code (instructions)"] --> Globals["Global/static data"]
  Code --> Consts["Read-only constants (e.g. string literals)"]
  Code --> Heap["Heap (dynamic memory)"]
  Code --> Stack["Stack (call frames)"]
```

This diagram shows typical process memory regions. Global/static variables and constants persist for the program duration [4]; the stack holds local (automatic) variables [5]; the heap is for `new` allocations (covered later).

- **Exercises:**

- *Easy:* Modify the program to greet the user by name (print `Hello, <your name>!`).
- *Easy:* Write a program that prints a multi-line message (use `"\n"` or multiple `<<` operations).
- *Medium:* Write a C++ program that computes and prints the sum of two integers (hard-code values).
- *Medium:* Intentionally introduce a syntax error (e.g. remove `;`) and observe the compiler error message (learn to read diagnostics).

- *Hard:* Create a simple ASCII art (e.g. a box or smiley) using `std::cout`.

- **Mini-Project:** *Interactive greeting:* Write a program that reads the user's name (using `std::cin`) and age, then prints a personalized greeting (e.g. "Hello, Alice (age 30)!"). This reinforces I/O and string handling basics.

- **Suggested Reading:** C++ *basic tutorial* (for example, learncpp.com's Introduction); Chapter 1 of Stroustrup's *Programming: Principles and Practice Using C++*; *cppreference*: `cppreference.com/iostream` (for `std::cout`, `std::cin`).

- **Checkpoint Quiz:** What is `main()`? What does `#include <iostream>` do? Name the four stages of compilation [6].

Session 2: Variables, Types & I/O

Objectives: Learn about C++ variable declarations, basic types (`int`, `double`, `char`, `bool`, `std::string`), initialization, and standard input/output. Understand `auto` for type deduction and the importance of type safety.

Key Concepts: Primitive types, `std::string` vs C-string, `cin/cout`, `std::endl`, integer vs floating-point, simple type conversions.

```
#include <iostream>
#include <string>
int main() {
    int age = 25;                // integer type
    double pi = 3.14159;         // floating-point type
    char c = 'A';                // single character
    bool flag = true;            // boolean type
    std::string name = "Alice"; // C++ string class

    std::cout << "Name: " << name << "\n";
    std::cout << "Age: " << age << ", Pi: " << pi << std::endl;

    // Input example
    std::cout << "Enter a number: ";
    int x;
    std::cin >> x;              // read integer from user
    std::cout << "You entered: " << x << std::endl;
    return 0;
}
```

- **Annotations:** We declare variables with type (e.g. `int age`). `std::string` (from `<string>`) represents text strings; it handles memory automatically. Use `std::cin` to read from standard input. Always validate input in real programs.

- **Pitfalls:** Uninitialized variables hold garbage; always initialize. Beware mixing types: e.g. assigning a `double` to an `int` truncates decimal (not an error but possible surprise). The `char` literal uses single quotes. C++ is statically typed.

- **Exercises:**

- *Easy:* Declare three variables (int, double, string) and print them.
- *Easy:* Read two integers from input and print their product.
- *Medium:* Swap two numbers without using a temporary variable (hint: arithmetic or `std::swap`).
- *Medium:* Use `auto` to declare a variable and initialize it with `5.5`; print its type at compile time (using `typeid`).

- *Hard:* Write a program that prompts for radius (double) and computes area of a circle. Use `M_PI` from `<cmath>` or define PI.

- **Mini-Project:** *Unit Converter:* Create a program that converts temperature between Celsius and Fahrenheit. Ask the user for a temperature and conversion direction, then output the converted value. This combines I/O, variables, and arithmetic.

- **Suggested Reading:** *cppreference* on fundamental types (e.g. `cppreference.com/types`), and on `std::cin` / `std::cout` [7]. Meyers, *Effective C++* (see item on initialization); Lippman et al., *C++ Primer* (Chapter on basic types).

Session 3: Control Flow (if, loops, switch)

Objectives: Master conditional statements (`if`, `else`, `switch`) and loops (`for`, `while`, `do-while`). Implement decision-making and repetition in programs.

Key Concepts: Boolean expressions, branch execution, loop constructs, break/continue.

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;

    if (n < 0) {
        cout << "Negative number!" << endl;
    } else if (n == 0) {
        cout << "Zero!" << endl;
    } else {
        cout << "Positive number!" << endl;
    }

    // Loop: calculate sum 1..n
    int sum = 0;
    for(int i = 1; i <= n; ++i) {
        sum += i;
    }
    cout << "Sum from 1 to " << n << " is " << sum << endl;

    return 0;
}
```

- **Annotations:** Conditional `if/else` checks conditions. `for` loop runs `i` from 1 to `n`. Braces `{}` define block scope. Use `==` for comparison, `=` for assignment.

- **Pitfalls:** Off-by-one errors in loops (e.g. using `<=` vs `<`). Forgetting to initialize loop counters or accumulating variables. Infinite loops if loop condition never fails. Always ensure loop termination.

- **Exercises:**

- *Easy:* Write a loop to print the first 10 natural numbers.
- *Easy:* Modify above program to print only even numbers up to `n`.
- *Medium:* Use a `while` loop to compute the factorial of `n` (n!).
- *Medium:* Use a `switch` statement to print the name of the day of week (1=Monday, ...).

- *Hard:* Implement a function that checks if a number is prime (loop to test divisibility). Return true/false.

- **Mini-Project:** *Guessing Game:* Write a program that generates a random number (1–100) and lets the user guess. Use a loop and give feedback "too low/high" until correct. Use `std::rand()` (include `<cstdlib>`) for simplicity.

- **Suggested Reading:** *cppreference* on flow control statements. A tutorial (like learncpp.com chapters on branches/loops) is also good. Focus on code examples rather than memory, since logic is key here.

Session 4: Functions & Program Flow

Objectives: Learn to organize code into functions. Understand call-by-value vs call-by-reference, parameters, return values. Explore the call stack and function activation records briefly.

Key Concepts: Function declaration/definition, parameters, `return`, scope of variables, header files, and simple recursion.

```
#include <iostream>
using namespace std;

// Function declaration (forward)
int add(int a, int b);

int main() {
    int x = 10, y = 20;
    int result = add(x, y);  // call by value
    cout << "10 + 20 = " << result << endl;

    return 0;
}

// Function definition
int add(int a, int b) {
```

```
    return a + b;
}
```

- **Annotations:** We declare `add` above `main` and define it below. Parameters `a,b` are *copied* from `x,y` (call-by-value). Changes to `a,b` inside `add` do not affect `x,y`.

- **Call Stack:** Each function call creates a new *stack frame* for local variables and return address [8]. When `add` returns, its frame is popped. Stack memory is limited, so very deep recursion can overflow (stack overflow).

- **Pitfalls:** Forgetting to declare functions (undefined reference linking error [9]). Off-by-value confusion: modifying parameters won't change originals. Recursive functions require base case to avoid infinite recursion.

- **Exercises:**

- *Easy:* Write a function `int square(int x)` that returns x*x, and use it in `main`.
- *Easy:* Change `add` to subtract; test negative results.
- *Medium:* Write a recursive function to compute factorial. (Mind base case!)
- *Medium:* Write an overloaded function (same name) for `add(double, double)` to add real numbers.

- *Hard:* Implement the Euclidean algorithm as a function `int gcd(int a, int b)` using recursion or loop.

- **Mini-Project:** *Calculator Module:* Create multiple functions for add, subtract, multiply, divide (taking two `double`s). In `main()`, prompt the user for two numbers and an operator, then call the appropriate function and display the result. Ensure division by zero is handled with an error message.

- **Suggested Reading:** Sections on functions in *C++ Primer*. Also see cppreference for functions and calling conventions. Understand the One Definition Rule (ODR) to avoid multiple-definition errors [10].

Session 5: Pointers & References

Objectives: Understand pointers (memory addresses) and references (aliases). Learn to use pointers to access dynamic memory and to create data structures. Emphasize the differences between stack vs heap storage.

Key Concepts: Pointer syntax (`int* p`), address-of `&` and dereference `*`. Reference syntax (`int& r`). Null pointers, pointer arithmetic basics.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int value = 42;
    int *p = &value;        // p holds address of value
    int &r = value;         // r is a reference (alias) to value

    cout << "value = " << value << endl;
    cout << "*p = " << *p << " (via pointer)" << endl;
    cout << "r = " << r << " (via reference)" << endl;

    *p = 100;               // change value through pointer
    cout << "value after *p = 100: " << value << endl;

    // NULL pointer example (nullptr in C++11+)
    int *q = nullptr;
    // *q would be undefined (segfault)

    return 0;
}
```

- **Annotations:** `int *p` declares a pointer to `int`. We assign it the address of `value` with `&value`. The expression `*p` accesses the integer at that address. `int &r = value` creates a reference alias.

- **Stack vs Heap:** Here `value` is on the stack (automatic). If we do `int *q = new int(5);`, that allocates on the heap (dynamic storage) [11]. The pointer `q` on stack holds the heap address. We must later `delete q;` to free it (covered in next session).

- **Pitfalls:** Dereferencing a null or uninitialized pointer causes undefined behavior (segfault). Dangling pointer: if you `delete` memory but still use the pointer, UB. Reference must be initialized (cannot be null or reseated).

- **Exercises:**

- *Easy:* Print the address of a variable using `&`.
- *Easy:* Swap two integers using pointers (function parameters `int* a, int* b`).
- *Medium:* Allocate an `int` on the heap using `new`, assign a value, print it, then `delete` it.
- *Medium:* Write a function that takes an array and its size by pointer and prints elements.

- *Hard:* Implement a function to reverse a C-style string (char array) *in place* using pointers.

- **Mini-Project:** *Dynamic Array:* Write a small wrapper that creates a dynamic array of `int` of user-specified size (`new int[size]`), reads values into it, computes the average, then frees the memory (`delete[]`). This reinforces `new[]` / `delete[]` and pointer arithmetic.

- **Suggested Reading:** *cppreference* on pointers and references. Also, Storage durations for static/automatic/dynamic [12]. A good tutorial is learncpp.com "Pointers" and "References" chapters.

Session 6: Dynamic Memory & RAII

Objectives: Delve into dynamic memory management. Understand manual allocation (`new/delete`), memory leaks, and the RAII idiom. Introduce smart pointers (`std::unique_ptr`, `std::shared_ptr`) for automatic resource management.

Key Concepts: `new` / `delete`, `new[]` / `delete[]`, heap memory, smart pointers, ownership.

```
#include <iostream>
#include <memory>
using namespace std;

int main() {
    // Manual allocation
    int *p = new int(10);       // allocate int on heap
    cout << "*p = " << *p << endl;
    delete p;                   // free memory

    // Smart pointer
    unique_ptr<int> up(new int(20));
    cout << "*up = " << *up << " (unique_ptr)" << endl;
    // No need to delete; up goes out of scope

    // shared_ptr example
    shared_ptr<int> sp1 = make_shared<int>(30);
    {
        shared_ptr<int> sp2 = sp1;  // shared ownership
        cout << "Count = " << sp1.use_count() << endl;
    } // sp2 destroyed, count decreases
    cout << "Count after sp2 destroyed: " << sp1.use_count() << endl;

    return 0;
}
```

- **Annotations:** Using `new int(10)` allocates on the heap [12]. Always `delete` what you `new`, else memory leaks. Prefer smart pointers (`std::unique_ptr`, `std::shared_ptr` in `<memory>`) which automatically delete when out of scope.

- **Pitfalls:** Mismatched `new[]` and `delete` (use `delete[]` for arrays). Forgetting `delete` leaks memory. Double-`delete` causes UB. Ownership ambiguity leads to leaks or premature frees. Smart pointers manage these issues but watch for cyclic `shared_ptr` references.

- **Exercises:**

- *Easy:* Allocate a `double` on heap, assign a value, then `delete` it.
- *Easy:* Allocate an array of 5 ints on the heap, fill with values, `delete[]` it.
- *Medium:* Create a `struct Node { int data; Node* next; };` and use `new` to build a simple singly-linked list of 3 nodes; then delete them manually.
- *Medium:* Rewrite a linked list using `std::unique_ptr<Node>` for `next` pointers to automate deletion.

- *Hard:* Explain why `unique_ptr` cannot be copied (compile-time error) but can be moved (use `std::move`). Demonstrate with code.

- **Mini-Project:** *Smart Pointer Practice:* Build a simple class `Matrix` that holds a dynamically allocated 2D array (use `new[]`). Implement constructor/destructor to allocate/free. Then rewrite it using `std::unique_ptr` (maybe a `unique_ptr<int[]>` for flat data). Compare safety.

- **Suggested Reading:** *cppreference* on dynamic memory and smart pointers. Meyers' *Effective Modern C++* (Item on smart pointers and move semantics). C++ Core Guidelines (Resource management; ownership).

Session 7: Arrays and Strings

Objectives: Explore C-style arrays, `std::array`, C-strings (`char[]`), and `std::string`. Highlight differences and pitfalls of C-strings (null terminator, buffer overflow). Use arrays in loops and pointer arithmetic.

Key Concepts: Static arrays (fixed-size), `std::vector` (brief mention for dynamic arrays), difference between `char[]` and `std::string`.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // C-style array
    int arr[5] = {1, 2, 3, 4, 5};
    for(int i = 0; i < 5; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // C-string (null-terminated)
    char name_cstr[] = "Bob";
    cout << "C-string: " << name_cstr << endl;
    cout << "Length: " << strlen(name_cstr) << endl; // cstring
```

```
    // std::string
    string name = "Charlie";
    cout << "std::string: " << name << " (length " << name.size() << ")" <<
endl;

    return 0;
}
```

- **Annotations:** `int arr[5]` allocates 5 ints on the stack. Access via `arr[index]`. `char name_cstr[] = "Bob";` creates an array of 4 chars (`'B','o','b','\0'`). The `strlen` function (from `<cstring>`) computes length up to the null terminator. `std::string` (from `<string>`) is safer and resizable.

- **Pitfalls:** Forgetting the `'\0'` in char arrays leads to garbage beyond. Array bounds not checked (writing beyond index causes undefined behavior). `std::string` is much safer for text.

- **Exercises:**

- *Easy:* Declare an array of 10 `double`s and initialize with values; print them.
- *Easy:* Read a word into a `std::string` (no spaces) and print its first and last character.
- *Medium:* Write a function that reverses a C-string in place (use two-pointer technique).
- *Medium:* Use `std::vector<int>` to read `n` numbers from input and compute their sum (introduces `vector`).
- *Hard:* Implement `strlen` yourself (loop until `'\0'`). Also implement `strcpy` (copy a C-string).

- **Mini-Project:** *Word Counter:* Read a full line of text (use `std::getline`) into a `std::string`, then count and output the number of words (separated by spaces). This uses `std::string` and basic parsing.

- **Suggested Reading:** C-strings pitfalls are discussed in many tutorials. *cppreference* on std::string and C-string functions. Stroustrup's *Programming* covers arrays and strings. Understand that using `std::string` is recommended in C++ over char arrays.

Session 8: Classes & Objects (Basics)

Objectives: Introduce object-oriented programming: define classes, create objects, and use constructors/destructors. Practice encapsulation and the `public`/`private` access specifiers.

Key Concepts: `class` and `struct`, member variables/functions, constructors (default, parameterized), destructors, `this` pointer, initializer lists.

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Person {
private:
    string name;
    int age;
public:
    // Constructor
    Person(const string &n, int a) : name(n), age(a) {
        cout << "Creating Person " << name << endl;
    }
    // Destructor
    ~Person() {
        cout << "Destroying Person " << name << endl;
    }
    void sayHello() const {
        cout << "Hi, I'm " << name << ", age " << age << endl;
    }
};

int main() {
    Person p1("Alice", 30);     // create on stack
    p1.sayHello();              // use method
    // Person *p2 = new Person("Bob", 25); // create on heap (show new/delete
usage)
    // delete p2;
    return 0;
}
```

- **Annotations:** `Person` has private data (`name`, `age`) and a public constructor and method. The constructor uses an initializer list `: name(n), age(a)`. Destructor runs at scope exit (here at program end) [13]. Calling `sayHello()` uses the object's data.

- **Pitfalls:** Forgetting to define a constructor for necessary initialization. If you use `new`, you must `delete` (else memory leak). Also the order of construction: bases first, then members in declaration order. If dynamic allocation fails (rare), `std::bad_alloc` is thrown.

- **Exercises:**

- *Easy:* Define a class `Point` with `int x,y`; write a constructor and a method to print coordinates.
- *Easy:* Create two `Point` objects and swap their coordinates (write a `swap()` method).
- *Medium:* Add a copy constructor to `Person` class that prints a message when copying. Observe when it's invoked (e.g. passing by value).
- *Medium:* Write a `Circle` class with `double radius`, including a constructor and a method for area (`return 3.14159 * radius * radius`).

- *Hard:* Prevent copying of a class by deleting the copy constructor and assignment operator (C++11: `ClassName(const ClassName&) = delete;`).

- **Mini-Project:** *Bank Account:* Design a `BankAccount` class with private `balance` (double) and public methods `deposit(double)`, `withdraw(double)`, and `getBalance()`. Ensure no negative balance (check in `withdraw`). Test it by creating an account and performing transactions.

- **Suggested Reading:** Lippman's *C++ Primer* Chapters on classes. *cppreference* on classes (including special member functions). The concept of object lifetime and RAII is reinforced (object lives from constructor end to destructor start [14] [15]).

Session 9: Inheritance & Polymorphism

Objectives: Cover class inheritance, `virtual` functions, and polymorphism. Learn how to define base and derived classes, override methods, and use dynamic dispatch.

Key Concepts: Single inheritance, `virtual` keyword, base pointer to derived object, abstract classes (pure virtual). Slicing issues.

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void speak() const { cout << "Animal sound" << endl; }
    virtual ~Animal() = default;  // virtual destructor
};

class Dog : public Animal {
public:
    void speak() const override { cout << "Woof!" << endl; }
};

class Cat : public Animal {
public:
    void speak() const override { cout << "Meow!" << endl; }
};

void makeSpeak(const Animal &a) {
    a.speak();
}

int main() {
    Dog d;
    Cat c;
    makeSpeak(d);  // dynamic dispatch: Woof!
    makeSpeak(c);  // Meow!
```

```
    return 0;
}
```

- **Annotations:** `Animal::speak()` is virtual, so calling `speak()` on an `Animal&` invokes the derived version (dynamic binding) [16]. We use `override` to ensure correct overriding. The base destructor is virtual to allow safe `delete` of derived objects via base pointer.

- **Pitfalls:** Omitting `virtual` leads to static binding (base version always called). Object slicing: assigning a derived object to a base object (not pointer/reference) cuts off derived parts. Always use references or pointers for polymorphism.

- **Exercises:**

- *Easy:* Derive a class `Bird` from `Animal` with its own `speak()`. Call it through a base pointer.
- *Easy:* Remove `virtual` from `speak()` and observe effect (the base version is called, not overridden).
- *Medium:* Create an abstract class `Shape` with a pure virtual `double area() const = 0;`. Derive `Rectangle` and `Circle` that implement `area()`.
- *Medium:* Demonstrate object slicing by writing `Animal a = Dog(); a.speak();`. Explain the result.

- *Hard:* Explain why the `Animal` destructor must be virtual when deleting derived objects through an `Animal*` pointer (memory leak or undefined behavior otherwise).

- **Mini-Project:** *Zoo Simulator:* Create several animals (`Dog`, `Cat`, plus your own e.g. `Cow`) and store pointers to them in a `std::vector<Animal*>`. Loop over them calling `speak()`. Then delete them properly. This shows polymorphism in a collection.

- **Suggested Reading:** *cppreference* on inheritance and virtual functions. Meyers' *Effective C++* (Item on polymorphic base class). Understand the **One Definition Rule (ODR)** when overriding methods [10].

Session 10: Templates & Generic Programming

Objectives: Introduce templates (function and class templates) for generic programming. Show how to write code that works for multiple types, reducing duplication.

Key Concepts: Template syntax `template<typename T>`, function templates, class templates, type inference vs explicit, template specialization (mention).

```
#include <iostream>
using namespace std;

// Function template
template <typename T>
T add(T a, T b) {
```

```
    return a + b;
}

// Class template
template <typename T>
class Box {
private:
    T value;
public:
    Box(T v) : value(v) {}
    T get() const { return value; }
};

int main() {
    cout << add<int>(2, 3) << endl;       // explicit template args
    cout << add(2.5, 3.1) << endl;        // deduced as double

    Box<string> b("Hello");
    cout << b.get() << endl;

    return 0;
}
```

- **Annotations:** `add` is a function template – works for any type `T` supporting `+`. Template arguments can be specified or deduced. `Box<T>` is a class template holding a value of type `T`.

- **Pitfalls:** Linking errors if templates are defined in `.cpp` (templates need to be visible at compile time: usually fully in headers). Type deduction can fail if implicit conversions needed.

- **Exercises:**

- *Easy:* Write a template function `swapValues(T &a, T &b)` that swaps two values.
- *Easy:* Write a template class `Pair<T1,T2>` that holds two values of possibly different types.
- *Medium:* Create a template specialization: specialize `add<string>` to concatenate strings.
- *Medium:* Explain why `template<typename T> void f(const T &x);` can bind to lvalues and rvalues. Try calling with an rvalue.

- *Hard:* What is the output and type for `add(2, 3.5)`? (Hint: template arguments deduce to a common type or cause ambiguity.)

- **Mini-Project:** *Generic Array Sum:* Write a function template that takes a pointer to an array of `T` and its size, and returns the sum (or concatenation) of its elements. Test with `int[]` and `double[]`. Use it in `main()`.

- **Suggested Reading:** *cppreference* on templates and std::vector for dynamic arrays. Vandevoorde & Josuttis, *C++ Templates* (for depth). Note: templates lead to **code generation** during compilation (instantiations).

Session 11: STL Basics – `std::vector`, `std::list`, Iterators

Objectives: Learn the Standard Template Library (STL) containers: dynamic array (`std::vector`), doubly-linked list (`std::list`), and use iterators. Emphasize writing generic code with these.

Key Concepts: `std::vector` usage (`push_back`, indexing), iterators (`begin()`, `end()`), range-based for loops (`for (auto &x : container)`), `std::list`.

```
#include <iostream>
#include <vector>
#include <list>
using namespace std;

int main() {
    vector<int> nums;
    nums.push_back(3);
    nums.push_back(1);
    nums.push_back(4);
    for (int x : nums) {              // range-based loop
        cout << x << " ";
    }
    cout << endl;

    list<string> words;
    words.push_back("hello");
    words.push_back("world");
    for (auto it = words.begin(); it != words.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    return 0;
}
```

- **Annotations:** `std::vector` is a dynamic array (resizes automatically). `std::list` is a linked list (efficient insert/erase). Range-based loops and iterators allow uniform access. Iterators generalize pointers.

- **Pitfalls:** Out-of-bounds if using `operator[]` incorrectly; use `at()` for bounds checking if needed. Inserting into a `std::vector` may invalidate iterators (because of reallocation). Understand `begin()/end()` semantics.

- **Exercises:**

- *Easy:* Read `n` integers into a `std::vector` and then print them backwards using an index loop.

- *Easy:* Use `std::list` of `char` and remove vowels from a string by copying its characters into the list and erasing vowels with `erase()`.
- *Medium:* Sort the `vector<int>` (include `<algorithm>` and use `std::sort(nums.begin(), nums.end())`). Print sorted results.
- *Medium:* Insert an element at the beginning of a `vector` (hint: `insert(begin(), value)`). What happens performance-wise?

- *Hard:* Use iterators to advance `std::list<int>` iterator by 3 positions (use `std::advance`).

- **Mini-Project:** *Shopping List:* Create a `std::vector<string>` of grocery items (add at least 5). Allow the user to mark items bought by entering an index; then remove that item from the list (use `erase`). Continue until list is empty, printing updated list each time.

- **Suggested Reading:** *cppreference* on `std::vector` and `std::list`. The concept of iterators is fundamental to STL (see container.iterators). Stroustrup or Lippman discuss the pros/cons of different containers.

Session 12: STL Algorithms & More Containers

Objectives: Introduce more STL components: associative containers (`std::map`, `std::set`) and common algorithms (`std::sort`, `std::find`, `std::binary_search`). Emphasize using `<algorithm>` functions on containers.

Key Concepts: `std::map` (ordered key-value store), `std::set` (ordered unique elements), algorithm library (sorting, searching, numeric algorithms).

```
#include <iostream>
#include <map>
#include <set>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    // Map example
    map<string,int> ageOf;
    ageOf["Alice"] = 30;
    ageOf["Bob"] = 25;
    for (auto &p : ageOf) {
        cout << p.first << " is " << p.second << endl;
    }

    // Set example
    set<int> nums = {4, 1, 3, 2, 4};
    for (int x : nums) {
        cout << x << " ";  // prints 1 2 3 4 (unique, sorted)
```

```
    }
    cout << endl;

    // Algorithm example
    vector<int> v = {5, 2, 8, 3};
    sort(v.begin(), v.end());
    if (binary_search(v.begin(), v.end(), 3)) {
        cout << "3 is in vector\n";
    }
    return 0;
}
```

- **Annotations:** `map<string,int>` maps names to ages. It stores keys sorted, values accessible by `operator[]`. `set<int>` holds unique sorted integers. Algorithms in `<algorithm>` work with iterators (as above).

- **Pitfalls:** `std::map::operator[]` creates a value if key not exist (initialize to default). Iterating a map goes in key order (not insertion order). `std::binary_search` requires sorted data. Using `std::find` vs container-specific `count` (maps/sets have `count()`).

- **Exercises:**

- *Easy:* Use `std::find` to locate a number in `std::vector<int>`. (Include `<algorithm>`).
- *Easy:* Given a `vector<int>`, use `std::count(v.begin(), v.end(), value)` to count occurrences of a number.
- *Medium:* Create a `map<char,int>` to count letter frequency in a string.
- *Medium:* Given a sorted `std::vector<string>`, use `std::binary_search` to check for a word.

- *Hard:* Explain the complexity (Big-O) of inserting into a `map`, a `vector`, and a `set` (hint: maps/sets are typically implemented as balanced trees, *vector* insertion may be linear).

- **Mini-Project:** *Phone Book:* Create a simple phonebook using `std::map<string,string>` where keys are names and values are phone numbers. Read name-number pairs from the user until an empty name is entered. Then allow the user to lookup phone numbers by name until they enter "exit".

- **Suggested Reading:** *cppreference* for `std::map`, `std::set`, and `<algorithm>` (e.g. `std::sort`). The STL documentation and *Effective STL* (Scott Meyers) cover use cases and pitfalls.

Session 13: Resource Management – Smart Pointers & Move Semantics

Objectives: Delve deeper into modern C++ resource management: smart pointers (`std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`), move semantics, and the Rule of Five. Understand `std::move` and when objects are moved vs copied.

Key Concepts: `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`; `std::move`; move constructors/assignment; `= delete`.

```
#include <iostream>
#include <memory>
#include <utility>
using namespace std;

struct Foo {
    Foo() { cout << "Foo constructed\n"; }
    ~Foo() { cout << "Foo destroyed\n"; }
    // Move constructor
    Foo(Foo&& other) noexcept { cout << "Foo moved\n"; }
    // Copy constructor
    Foo(const Foo&) { cout << "Foo copied\n"; }
};

int main() {
    // unique_ptr and move
    unique_ptr<Foo> p1 = make_unique<Foo>();
    // unique_ptr<Foo> p2 = p1; // ERROR: can't copy unique_ptr
    unique_ptr<Foo> p2 = move(p1);  // OK: transfer ownership
    if (!p1) cout << "p1 is null after move\n";

    // shared_ptr and weak_ptr
    auto sp1 = make_shared<Foo>();
    cout << "Use count: " << sp1.use_count() << endl;
    shared_ptr<Foo> sp2 = sp1;
    cout << "Use count: " << sp1.use_count() << endl;
    weak_ptr<Foo> wp = sp1;  // does not affect count

    // Move semantics with objects
    Foo a;
    Foo b = move(a);  // invokes move constructor

    return 0;
}
```

- **Annotations:** `std::move(p1)` converts to rvalue allowing move constructor/assignment. After moving a `unique_ptr`, the source is null. `std::weak_ptr` doesn't keep object alive (`use_count()` unchanged) and must be locked to `shared_ptr` to use.

- **Pitfalls:** Accidentally copying `unique_ptr` (deleted compile-time) vs moving. Managing cycles: two `shared_ptr` referencing each other never free; use `weak_ptr` to break cycles. Incorrect use of `move`: after `Foo b = move(a);`, `a` is in a valid but unspecified state (generally "moved-from").

- **Exercises:**

- *Easy:* Explain what happens if you call `delete` on a `nullptr`. (It's safe/no-op.)
- *Easy:* Try copying a `unique_ptr` and observe the compiler error.
- *Medium:* Write a class `Resource` that acquires (e.g., prints "Acquired") in constructor and releases (prints "Released") in destructor. Use it with `unique_ptr` and `shared_ptr`, and observe when destructor runs.
- *Medium:* Create two `shared_ptr<int>` that form a cyclic reference (A holds B, B holds A) and explain why this leaks. Convert one to `weak_ptr` to fix it.

- *Hard:* Given `vector<Foo> v; v.push_back(Foo());`, explain whether `Foo` is copied or moved into the vector. (Answer: C++11+ uses move constructor if available [17].)

- **Mini-Project:** *Unique vs Shared:* Write a program that creates a `unique_ptr<Foo>` and a `shared_ptr<Foo>`. Transfer ownership of the `unique_ptr` to a function (take `unique_ptr<Foo>` by value). In a second part, create several `shared_ptr<Foo>` copies and display the use count at each step. This illustrates ownership semantics.

- **Suggested Reading:** *cppreference* on smart pointers and move semantics. Herb Sutter's talks or "Essentials of Modern C++" cover `std::move`. Meyers' *Effective Modern C++* (Items on move semantics, smart pointers) is authoritative. Note the Rule of Five for managing resources in classes.

Session 14: Modern C++ Features (auto, lambdas, range-for)

Objectives: Introduce convenient modern syntax and features from C++11 onward: `auto` type deduction, lambda expressions, range-based `for`, `constexpr`, etc. Emphasize writing clearer and safer code.

Key Concepts: `auto` for variable declarations, lambda functions (`[capture](args){...}`), `std::function`, `std::async` (teaser), `constexpr`.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    auto x = 5.5;           // double
    auto s = "hi";         // const char*

    vector<int> nums = {3,1,4,1,5};
    sort(nums.begin(), nums.end(), [](int a, int b) {
        return a < b;     // lambda comparator
    });

    for (auto n : nums) {  // range-for with auto
        cout << n << " ";
```

```
    }
    cout << endl;

    // lambda capturing by value and reference
    int factor = 2;
    auto multiply = [factor](int v) {
        return factor * v;
    };
    cout << "5 * factor = " << multiply(5) << endl;

    return 0;
}
```

- **Annotations:** `auto` deduces type from initializer. Lambda `[factor](int v){...}` captures `factor` by value. Range-based `for(auto n: nums)` iterates all elements (simpler than index loop). These features reduce verbosity.

- **Pitfalls:** Overuse of `auto` can hurt readability (type unclear). Capturing variables by reference in lambdas requires caution (dangling reference if the lambda outlives them). Lambdas can also capture `this` in classes (`[this]`).

- **Exercises:**

- *Easy:* Use `auto` in a for loop: `for(auto it = v.begin(); it != v.end(); ++it)`. What type is `it`?
- *Easy:* Write a lambda that takes two ints and returns their product. Use it to multiply two numbers.
- *Medium:* Given `vector<int> nums`, use `std::count_if` with a lambda to count how many elements are even.
- *Medium:* Modify the above lambda to capture a local threshold by reference, and count how many elements exceed that threshold.

- *Hard:* Explain what `constexpr` means for functions and variables. (Simple answer: can be evaluated at compile time if given constant inputs). Write a `constexpr int square(int x) { return x*x; }` and use it to initialize a `constexpr int`.

- **Mini-Project:** *Lambda Filter:* Read a list of integers into a vector. Ask the user for a predicate (e.g. "only even" or ">5"). Use `std::copy_if` with a lambda to filter the list into a new vector, then print the filtered results.

- **Suggested Reading:** *cppreference* on lambda expressions and `auto` and range-based for. Effective Modern C++ (Meyers) covers these extensively (Item on lambdas). CppCon talks by Jason Turner also give good insights.

Session 15: Concurrency Basics (Threads and Mutexes)

Objectives: Introduce multithreading in C++: `std::thread`, `std::mutex`, and the basics of synchronizing threads. Explain why concurrency is needed (performance) and common pitfalls.

Key Concepts: `std::thread`, launching threads, passing data, race conditions, locks (`std::mutex`, `lock_guard`), atomics (`std::atomic`), data races (undefined behavior).

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

mutex mtx;   // global mutex for syncing output

void printMessage(const string &msg) {
    lock_guard<mutex> lock(mtx); // lock until end of scope
    cout << msg << endl;
}

void count(int id) {
    for (int i = 0; i < 5; ++i) {
        printMessage("Thread " + to_string(id) + ": " + to_string(i));
        this_thread::sleep_for(chrono::milliseconds(100));
    }
}

int main() {
    thread t1(count, 1);   // start thread 1
    thread t2(count, 2);   // start thread 2

    // Wait for threads to finish
    t1.join();
    t2.join();

    cout << "Both threads finished." << endl;
    return 0;
}
```

- **Annotations:** We create two threads running `count(1)` and `count(2)`. `printMessage` uses a `mutex` to prevent interleaved output. `sleep_for` simulates work. We call `join()` on threads so `main` waits.

- **Pitfalls:** Race conditions occur if threads modify shared data without locks (undefined behavior). Deadlocks if two mutexes locked in different orders. Always use RAII (`lock_guard`) for mutexes to avoid forgetting `unlock()`. Overuse of locks may degrade performance.

- **Exercises:**

- *Easy:* Create two threads that each print numbers 1–3 (without synchronization). Observe interleaved output.
- *Easy:* Add a `mutex` and `lock_guard` to synchronize printing so outputs from each thread don't mix mid-line.
- *Medium:* Demonstrate a race condition by having two threads increment a shared `int counter` 1000 times each without locking, then with `atomic<int>`. Compare results.
- *Medium:* Use `std::this_thread::sleep_for` to have one thread wait for 1 second, another for 2 seconds, and show the asynchronous execution.

- *Hard:* Explain what happens if you `detach()` a thread vs `join()`. Write code where the main thread exits before detached thread finishes (undefined behavior or process termination?).

- **Mini-Project:** *Threaded Summation:* Split a large array of integers into two halves. Launch two threads, each summing one half, and store results in shared variables. Use `std::mutex` or `std::atomic<int>` to safely accumulate into a global sum. Compare performance with single-threaded sum for a large array.

- **Suggested Reading:** Concurrency tutorial on cppreference. C++11 introduced `<thread>` and `<mutex>` [18]. Anthony Williams' *C++ Concurrency in Action* is the authoritative book. Learn about the C++ memory model when possible.

Session 16: Concurrency Advanced (Futures, Async, Atomics)

Objectives: Explore higher-level concurrency primitives: `std::future` / `std::async`, `std::thread` pools concept, `std::condition_variable`, and atomic operations (`std::atomic`). Discuss concurrency patterns and thread safety.

Key Concepts: `std::async` for launching tasks, getting return values (`std::future`), `std::atomic<T>` for lock-free shared data, `std::condition_variable` for thread coordination.

```
#include <iostream>
#include <future>
#include <atomic>
#include <vector>
using namespace std;

int square(int x) {
    return x * x;
}
```

```
int main() {
    // std::async example
    future<int> fut = async(launch::async, square, 10);
    cout << "Square of 10 is " << fut.get() << endl;

    // Atomic example
    atomic<int> count{0};
    const int N = 100000;
    auto inc = [&count]() {
        for (int i = 0; i < N; ++i) {
            count.fetch_add(1, memory_order_relaxed);
        }
    };
    thread t1(inc), t2(inc);
    t1.join(); t2.join();
    cout << "Final count: " << count << endl;
    return 0;
}
```

- **Annotations:** `std::async` runs `square(10)` possibly on a new thread and returns a `std::future<int>`. Calling `get()` waits and retrieves the result. `std::atomic<int>` avoids data races without explicit locks for simple increments.

- **Pitfalls:** Futures may still block `main()` if not used carefully. Incorrect use of `condition_variable` can cause missed signals or deadlocks (not shown above due to complexity). Even with atomics, there are memory ordering considerations (we used `memory_order_relaxed` safely here for increments).

- **Exercises:**

- *Easy:* Launch multiple `std::async` tasks to compute factorial of different numbers, then retrieve and print results.
- *Easy:* Show what happens if you call `future.get()` twice on the same future (throw exception).
- *Medium:* Write code using `std::promise<int>` and `std::future<int>` explicitly to send data from one thread to another.
- *Medium:* Use an `std::condition_variable` to have one thread wait for a flag set by another thread (e.g., producer/consumer with a boolean ready).

- *Hard:* Explain the difference between `memory_order_relaxed`, `memory_order_acquire/release`, and why atomic increments may still not need the strongest ordering.

- **Mini-Project:** *Parallel Sum with async:* Given a large `std::vector<int>`, divide it into 4 chunks and launch 4 async tasks (or threads) to compute partial sums, then combine them. Compare performance with single-threaded sum.

- **Suggested Reading:** *cppreference* on `std::future` and `std::atomic`. The concept of the C++ memory model is deeper, but this session can reference the idea of relaxed vs strict ordering. In-depth coverage in *C++ Concurrency in Action* by Williams.

Session 17: Metaprogramming (constexpr, SFINAE, Concepts)

Objectives: Introduce compile-time programming: `constexpr` functions and variables, simple template metaprogramming (type traits, SFINAE), and C++20 Concepts. Show how computation and type checks can happen at compile time.

Key Concepts: `constexpr` for compile-time evaluation, template specialization for traits, SFINAE (Substitution Failure Is Not An Error), `static_assert`, C++20 `concept` keyword.

```
#include <iostream>
#include <type_traits>
using namespace std;

// constexpr function
constexpr int factorial(int n) {
    return (n <= 1) ? 1 : (n * factorial(n - 1));
}

// template metaprogramming: type trait
template <typename T>
struct is_integral_type {
    static constexpr bool value = false;
};
template <>
struct is_integral_type<int> {
    static constexpr bool value = true;
};

int main() {
    constexpr int f5 = factorial(5);  // computed at compile time
    cout << "5! = " << f5 << endl;

    static_assert(is_integral_type<int>::value, "int should be integral");
    static_assert(!is_integral_type<double>::value, "double not integral");

    // C++20 concept (if available)
    #if __cpp_concepts >= 201907L
    concept Integral = is_integral_type<int>::value; // simplification
    static_assert(Integral, "Integral concept holds for int");
    #endif
```

```
    return 0;
}
```

- **Annotations:** `factorial` marked `constexpr` means `f5` is evaluated at compile time (if compiler chooses). `is_integral_type` mimics `std::is_integral`; using `static_assert` checks conditions during compilation. C++20 Concepts allow cleaner constraints (not shown fully due to complexity).

- **Pitfalls:** Not all code can be `constexpr` (only certain operations allowed). Beware infinite recursion in `constexpr` (compile error). SFINAE errors can be hard to decipher; the simple trait above avoids errors by specialization.

- **Exercises:**

- *Easy:* Make a `constexpr` function that returns the larger of two ints. Use it to declare a `constexpr int`.
- *Easy:* Use `static_assert` to check a property (e.g. `static_assert(sizeof(int) >= 4, "int is too small");`).
- *Medium:* Write a template struct `is_pointer<T>` that has a static `value` true if `T` is a pointer type, false otherwise (specialize for `T*`).
- *Medium:* Show how SFINAE works: write two overloaded template functions `foo`: one enabled for integral types, one for other types, using `std::enable_if`.

- *Hard:* (Optional) If using C++20, define a concept `Addable<T>` that requires `T + T` to be valid. Use it to write a template function that only works on addable types.

- **Mini-Project:** *Compile-Time Math:* Create a `constexpr` template-based compile-time power function `constexpr int ipow(int base, int exp)` that computes `base^exp`. Use `static_assert` to check that `ipow(2, 10) == 1024`.

- **Suggested Reading:** *cppreference* on `constexpr` and type traits. Jaap Suter's *C++ Templates* or Mandated by Standard metaprogramming sections. Even though template metaprogramming can get complex, focus on simple compile-time constants and `static_assert`.

Session 18: Data Structures I (Complexity, Containers Review)

Objectives: Introduce algorithmic complexity (Big-O notation) and review key data structures from both STL and custom: dynamic arrays, linked lists, stacks, queues, trees (conceptually). Emphasize how choice of structure affects performance.

Key Concepts: Time complexity of common operations, conceptual understanding of data structures.

- **Algorithms Complexity:**
- **Array (`std::vector`):** Access O(1), search O(n), insert at end amortized O(1), insert at front O(n).
- **Linked List (`std::list`):** Insertion/deletion O(1) (given position), search O(n).

- **Stack/Queue**: Typically O(1) push/pop (using `std::stack` / `std::queue`, which wrap `vector` or `deque`).
- **Map/Set**: O(log n) operations (balanced tree).
- **Hash Table (e.g. `std::unordered_map`)**: average O(1) lookup (not covered deeply, but mention).
- **Example (Table):**

Structure	Average-case Access	Search	Insert/Erase
`std::vector`	O(1) (by index)	O(n)	O(1) at end, O(n) at arbitrary position
`std::list`	O(n) (by index)	O(n)	O(1) (once positioned)
`std::map`	N/A (no index)	O(log n)	O(log n)
`std::unordered_map`	N/A	O(1) avg	O(1) avg

- **Visual – Comparison Diagram:**

```
graph LR
    Vectors["std::vector"] ---|Access by index| VTime["O(1)"]
    Vectors ---|Search| VSearch["O(n)"]
    Lists["std::list"] ---|Traverse| LTime["O(n)"]
    Maps["std::map"] ---|Lookup| MTime["O(log n)"]
```

- **Exercises:**

- *Easy:* Compute by hand: If `vector<int>` has size 1000, how many steps to find an element by scanning vs. `map<int,int>` lookup?
- *Easy:* Implement a simple stack using `std::vector`: push (push_back) and pop (pop_back).
- *Medium:* Use `std::queue<int>` to read a list of numbers and then output them in FIFO order.
- *Medium:* Explain why `std::vector` might reallocate (what happens internally when capacity exceeded).
- *Hard:* Suppose you need to check existence of many random numbers quickly. Which is better: `std::set<int>` or sorted `std::vector<int>` with binary search? Explain trade-offs.
- **Mini-Project:** *Library of Structures:* Implement a simple dynamic array class `MyVector` that supports `push_back`, `operator[]`, and resizing. Compare it to `std::vector` by usage. This reinforces understanding of underlying array mechanics.

- **Suggested Reading:** *Big-O* references (e.g. GeeksforGeeks Stack vs Heap for memory, and algorithm complexity overview. Sedgewick's books, or CLRS (introduction). *cppreference* has complexity notes in container descriptions (see Complexity section of container pages).

Session 19: Algorithms I (Sorting, Searching, BFS/DFS)

Objectives: Cover fundamental algorithms: sorting (e.g. QuickSort, MergeSort) and searching (binary search) and basic graph traversal (Breadth-First Search, Depth-First Search). Discuss when to use which.

Key Concepts: Sorting algorithms (their complexities, `std::sort` uses Introsort), binary search on sorted data, BFS/DFS on graph (or tree) structures.

- **Sorting:**
- Common sorts: *QuickSort* (average O(n log n), worst O(n^2)), *MergeSort* (O(n log n) guaranteed), *HeapSort* (O(n log n)).
- `std::sort` typically implements *Introsort* (QuickSort + HeapSort fallback) for average O(n log n).
- **Binary Search:** For sorted array, search is O(log n). Use `std::binary_search` or `std::lower_bound`.
- **BFS/DFS:** Traverse graph in breadth-first or depth-first manner (both O(V+E)). BFS uses a queue, DFS uses recursion or stack.
- **Exercises:**
- *Easy:* Use `std::sort` on a vector of strings and print them alphabetically.
- *Easy:* Use `std::binary_search` to check if a number exists in a sorted `vector<int>`.
- *Medium:* Implement **bubble sort** on an array (O(n^2)) and observe performance on small arrays.
- *Medium:* Write a recursive DFS for a binary tree (structures and pointers needed).
- *Hard:* Given a large dataset (e.g. 1 million ints), why is `std::sort` much faster than bubble sort? (Explain time complexity effect.)
- **Mini-Project:** *Maze Solver:* Represent a maze as a 2D grid of 0 (open) and 1 (wall). Use BFS to find the shortest path from start to end (queue-based graph search). This uses graph traversal concepts.
- **Suggested Reading:** *cppreference* on `std::sort` and `std::binary_search`. GeeksforGeeks has articles on BFS/DFS (the logic is general CS). *The Algorithm Design Manual* by Skiena or CLRS for detailed understanding.

Session 20: Algorithms II (Graphs, Dynamic Programming, Greedy)

Objectives: Deepen algorithm practice: graph algorithms (Dijkstra's shortest path, if time allows), dynamic programming examples (e.g. knapsack, Fibonacci with memoization), and greedy algorithms (e.g. coin change, activity selection).

Key Concepts: Graph representation (`adjacency list`), DP vs recursive, greedy strategy, problem-solving patterns.

- **Graph:** Introduce representing a graph as `vector<vector<pair<int,int>>>` for weighted edges. Use Dijkstra's algorithm (priority queue) for shortest path in O(E log V).

- **Dynamic Programming:** Show how caching previous results reduces complexity (Fibonacci vs DP). E.g. compute Fibonacci numbers iteratively or with memo.

- **Greedy:** Examples like coin change (min number of coins, works for canonical coin sets), or interval scheduling.

- **Exercises:**

- *Easy:* Implement Fibonacci with recursion and then with a loop or memoization. Compare running time.
- *Easy:* For an array, write a greedy algorithm: repeatedly pick the maximum element and subtract it until zero (not optimal change).
- *Medium:* Implement a simple version of Dijkstra's (unweighted, use BFS for shortest path).
- *Medium:* Write a DP solution for the 0/1 knapsack problem (small n, W).

- *Hard:* Solve the "Coin Change" problem: given coin denominations and amount, find minimum coins. First try greedy, then DP. Explain why greedy fails for arbitrary sets.

- **Mini-Project:** *Traveling Salesman (TSP) Approximation:* Given a list of cities with distances (metric), implement the nearest-neighbor heuristic (a greedy approach to TSP). Compare path length to brute-force (small N) or known optimal for a tiny example.

- **Suggested Reading:** *cppreference* or Wikipedia for Dijkstra's algorithm (no direct page, but concept). *Competitive programming* sources (CP-algorithms, GeeksforGeeks) for DP/graph code. It's useful to reinforce complexity and algorithm design skills for programming contests.

Session 21: Build Systems, Debugging, and Tools

Objectives: Teach practical tools for development: compilers, build systems (Make, CMake), debuggers, and sanitizers. Show how to compile multi-file projects and debug errors.

Key Concepts: Compiler flags, linking libraries, Makefile/CMakeLists, gdb/LLDB usage, sanitizers (AddressSanitizer, UndefinedBehaviorSanitizer), IDE options.

- **Compilers:**
- **GCC/Clang:** Use `-std=c++11/14/17/20/23`, `-Wall -Wextra` for warnings, `-g` for debug info. For example:

```
g++ -std=c++20 -Wall -Wextra -g main.cpp foo.cpp -o app
```

- **MSVC (cl):** Use `/std:c++latest` or `/std:c++17`, `/EHsc`, and `/W4` for warnings.

- **Build Systems:**

- **Make:** Write a `Makefile` with targets and dependencies. Example snippet:

```
CXX = g++
CXXFLAGS = -std=c++20 -Wall -Wextra -g
OBJS = main.o module.o utils.o

app: $(OBJS)
    $(CXX) $(CXXFLAGS) $(OBJS) -o app

%.o: %.cpp
    $(CXX) $(CXXFLAGS) -c $< -o $@
```

- **CMake:** Use a `CMakeLists.txt` to configure cross-platform build. Example excerpt:

```
cmake_minimum_required(VERSION 3.10)
project(MyApp LANGUAGES CXX)
add_executable(app main.cpp module.cpp)
target_compile_features(app PRIVATE cxx_std_20)
```

  Then `mkdir build; cd build; cmake ..; make`.

- Compare: **Make** is simple and Unix-centric; **CMake** is portable, generates Makefiles or project files.

- **Debuggers:**

- **gdb (Linux):** `gdb ./app` then commands `break main`, `run`, `next`, `print var`, `backtrace`, etc.
- **LLDB (macOS):** Similar to gdb.
- **Visual Studio Debugger (Windows):** Graphic breakpoints, watches.

- **Valgrind (Linux, for memory checking)** and **Sanitizers:** compile with `-fsanitize=address,undefined` to catch out-of-bounds, use-after-free, etc.

- **Pitfalls:** Linking errors: missing `.cpp` files in build, forgetting to link libraries (e.g. `-lpthread` for threading on some systems). Use `-L/path -lname` to link custom libs. Understand error codes.

- **Exercises:**

- *Easy:* Write a small multi-file project (`a.cpp`, `b.cpp`) and a Makefile to build it.
- *Easy:* Introduce an intentional bug (e.g. divide by zero) and run with AddressSanitizer (`g++ -fsanitize=address`). Observe output.

- *Medium:* Use a debugger to step through code: set a breakpoint, inspect variables, step into function.
- *Medium:* Create a simple `CMakeLists.txt` for an existing project.

- *Hard:* Enable `-Werror` (treat warnings as errors) and fix the warnings in a given code. Discuss why code should have minimal warnings.

- **Mini-Project:** *Build Automation:* Use CMake to create a build for a small library and executable. For instance, library `mylib` in one directory and `main.cpp` linking against it. Demonstrate a clean build and running tests.

- **Suggested Reading:** Official docs: CMake Tutorial, GNU Make manual, and *cppreference*'s notes on compilation pipeline. Tools like Valgrind and sanitizers are widely documented online.

Session 22: Final Project & Review; Assessment

Objectives: Reinforce learning via a capstone project that integrates multiple concepts. Review key points from all sessions. Conduct a comprehensive quiz and discuss further learning resources and career paths.

Key Concepts: Project design, integration, debugging, code style. Self-assessment and future directions.

- **Project:** Propose a larger project, e.g., a text-based game or a mini-application combining data structures, classes, and algorithms. Example: *"Library Management System"* – use classes for `Book`, `Member`, use containers to store inventories, file I/O for persistence, etc. Or *"Graph Visualizer"* using an adjacency list representation and pathfinding algorithms.

- **Review Topics:**

- Program structure, compilation/linking, memory (stack vs heap).
- Language syntax and core concepts (types, control flow, functions).
- OOP (classes, inheritance, polymorphism).
- Generic programming (templates, STL).
- Modern C++ (lambdas, smart pointers, concurrency).
- DSA algorithms (sorting, searching, graph/DP basics).

- Tools and best practices (build systems, debugging, code style).

- **Assessment:**

- Conduct a **quiz/checklist** covering major topics (e.g. "What is undefined behavior?", "Explain RAII", "What does `constexpr` mean?", etc.).
- Encourage peer review of code, automated tests for submitted exercises, and personal progress logs.

- Metrics: number of problems solved, projects completed, and self-assessed comfort levels per topic.

- **Future Resources:** Provide a curated list of references by topic (see below). Encourage joining C++ communities (stackoverflow, reddit/r/cpp, ISO C++ forum) and continuous practice (e.g. coding challenges).

- **Encouragement:** Becoming proficient takes practice. Use this curriculum as a foundation and continue building projects. Remember that C++ expertise is highly valued in systems, game development, finance, and more [19].

Additional Resources and References

- **Official References:**
- *ISO C++ Standard:* (via isocpp.org or WG21 papers [20] [21]).
- **cppreference.com:** Comprehensive reference for C++ language and library (e.g., language, containers, algorithm) [22] [23].

- **C++ Core Guidelines:** Best practices (by Bjarne Stroustrup et al.) – GitHub: https://github.com/isocpp/CppCoreGuidelines.

- **Books:**

- B. Stroustrup, *"A Tour of C++"* (overview).
- L. Lippman et al., *"C++ Primer (5th ed.)"* (detailed intro).
- S. Meyers, *"Effective Modern C++"* (C++11/14 best practices).
- H. Schildt, *"C++: The Complete Reference"* (comprehensive).

- A. Williams, *"C++ Concurrency in Action"* (for threads).

- **Online Tutorials:**

- LearnCpp.com (especially good for beginners), GeeksforGeeks C++ sections.
- CompilerSutra C++ tutorials (outline at CompilerSutra [24] [25]).

- *cppreference*'s specialized pages for in-depth details.

- **Tools:**

- **Compilers:** GCC (GNU C++), Clang/LLVM, MSVC (Visual Studio). See GCC Standards Support [21] and Clang C++ Status [26].
- **Build Systems:** CMake (cross-platform), GNU Make, Ninja.
- **Debuggers:** GDB (Linux), LLDB (macOS), Visual Studio Debugger (Windows).
- **Sanitizers:** AddressSanitizer (`-fsanitize=address`), UndefinedBehaviorSanitizer (`-fsanitize=undefined`), ThreadSanitizer (`-fsanitize=thread`).

- **IDEs:** Visual Studio, CLion, VS Code (with C++ extensions), Code::Blocks, Xcode.

- **Standards Table (C++ Versions):**
| Standard | Year | Key Features | Compiler Support Notes |

|---------|-------|-----------|-----------| | C++98/03 | 1998/2003 | Legacy base, STL | Supported by all (select `-std=c++98/gnu++98`) | | C++11 | 2011 | auto, lambdas, threads | Fully supported from GCC 4.8, Clang 3.3, VS 2015 [27] | | C++14 | 2014 | Refinements (auto return, etc.) | Supported from GCC 5, Clang 3.4, VS 2015.3 | | C++17 | 2017 | std::optional, filesystem, structured bindings | Mature support in GCC 9+, Clang 5+, VS 2017 [27] | | C++20 | 2020 | Concepts, ranges, coroutines, modules | GCC 10+ (default in 16) [21], Clang 10+, VS 2019/2022 | | C++23 | 2023 | Extended consteval, etc. | Emerging support (GCC 12+, Clang 14+) |

- **Compiler Comparison:** Generally, **GCC** and **Clang** are widely used on Linux/Mac; **MSVC** is standard on Windows. All major compilers now support C++17 well; C++20 support is near-complete (check cppreference compiler support for details).

- **Visualization Tools:** Use tools like Compiler Explorer for studying generated assembly. Profilers (gprof, perf) and static analysis (cppcheck, clang-tidy) can improve code.

Mermaid Diagrams: (example visuals used above)

```
graph LR
    A[Source (.cpp)] --> B[Compiler]
    B --> C[Object (.o)]
    C --> D[Linker]
    D --> E[Executable]
```

This shows the **compilation pipeline**: preprocessing/compilation, assembly, and linking to produce a runnable binary [28] [2].

```
graph LR
    C98[C++98] --> C11[C++11] --> C14[C++14] --> C17[C++17] --> C20[C++20] -->
C23[C++23]
```

This simple timeline indicates the evolution of the C++ standard (years 1998, 2011, 2014, 2017, 2020, 2023). Each standard introduced key features (e.g. C++11 added lambdas and `auto`, C++20 adds concepts and ranges).

```
graph TB
    A[std::vector] -->|access O(1)| B[std::map]
    A -->|search O(n)| C[std::list]
    D[std::unordered_map] -->|lookup ~O(1)| B
    C -->|insert/erase O(1)| A
```

A conceptual diagram of data-structure performance (arrows denote comparative operations; e.g. unordered_map offers near O(1) lookup, maps O(log n) etc., as summarized in the session on DSA).

Each diagram is meant to reinforce concepts (memory layout, C++ versions, complexity).

Conclusion: This curriculum, with progressively deeper sessions, exercises, and projects, aims to transform a C++ novice into a confident programmer with a solid foundation in both language features and practical programming techniques. All information is drawn from up-to-date authoritative sources, and each topic is backed by reference citations as indicated above (e.g. CompilerSutra roadmap [24] [25], cppreference for language rules [22] [23]). Keep this guide and references handy as you learn, practice regularly, and build your skills. Good luck on your C++ journey!

---

[1] [3] [7] [13] A General Overview of What Happens Before main() - Embedded Artistry
https://embeddedartistry.com/blog/2019/04/08/a-general-overview-of-what-happens-before-main/

[2] [9] [10] [16] C++ Linking Process: How Object Files Become an Executable | A Practical Guide
https://www.studyplan.dev/cmake/cpp-object-files-and-linking

[4] [5] [11] [12] [15] [22] Storage class specifiers - cppreference.com
https://en.cppreference.com/w/cpp/language/storage_duration.html

[6] [28] passlab.github.io
https://passlab.github.io/ITSC3181/notes/lecture02_CompilationAssemblingLinkingProgramExecution.pdf

[8] [14] [17] Object Lifetime | KDAB
https://www.kdab.com/object-lifetime/

[18] [19] [24] [25] C++ Programming Curriculum (From Zero to Advanced) | CompilerSutra
https://www.compilersutra.com/docs/c++/cpp-learning-roadmap/

[20] [21] [27] C++ Standards Support in GCC - GNU Project
https://gcc.gnu.org/projects/cxx-status.html

[23] Undefined behavior - cppreference.com
http://en.cppreference.com/w/cpp/language/ub.html

[26] Clang - C++ Programming Language Status
https://clang.llvm.org/cxx_status.html